

Exercício 7: Classes Abstratas e Interfaces

Parte 1: Questões Conceituais de Fixação

1. **Classes Abstratas:** Por que o Java impede a instanciação direta de uma classe abstrata através do operador `new`? O que acontece se uma subclasse concreta herdar de uma classe abstrata e não implementar algum de seus métodos abstratos?
2. **Membros Estáticos (`static`):** Explique a diferença de comportamento entre um atributo de instância e um atributo estático (`static`). Em uma hierarquia de classes abstratas, para quais finalidades membros estáticos são comumente utilizados (por exemplo, contadores globais ou constantes de configuração)?
3. **Interfaces vs. Classes Abstratas:** Qual a diferença essencial entre estender uma classe abstrata (`extends`) e implementar uma interface (`implements`)? Por que uma interface permite que classes pertencentes a hierarquias totalmente distintas compartilhem o mesmo contrato de métodos?

Parte 2: Questões Práticas de Programação

Questão 1: Sistema de Gestão de Frota e Licenciamento (Classes Abstratas e Membros Estáticos)

Crie um projeto chamado `atividade_abstrata_estatico`.

Desenvolva uma aplicação para controlar e calcular as taxas de licenciamento de veículos em uma empresa de transporte.

4. **Crie a classe abstrata `Veiculo`:**
 - **Atributos privados de instância:** `placa` (String), `modelo` (String), `valorBase` (double).
 - **Atributo estático privado:** `totalVeiculosCadastrados` (int), inicializado em 0. Deve ser incrementado no construtor a cada novo veículo criado.
 - **Constante estática pública:** `TAXA_AMBIENTAL = 0.02` (double).
 - **Métodos:**
 - Construtor para inicializar os atributos de instância e incrementar o contador estático.
 - Métodos *getters* e *setters* para os atributos privados.
 - Método estático `public static int getTotalVeiculosCadastrados()`, que retorna o total de veículos instanciados.
 - Método concreto `public double calcularTaxaAmbiental()`, que retorna `valorBase * TAXA_AMBIENTAL`.
 - Método abstrato `public abstract double calcularLicenciamento()`.
5. **Crie a subclasse concreta `CarroPasseio` (herda de `Veiculo`):**
 - **Atributo privado:** `potenciaMotor` (double).

- Construtor que recebe os dados do veículo e a potência do motor (reutilizando o construtor da superclasse via **super**).
 - Sobrescreva o método **calcularLicenciamento()**: O valor do licenciamento é calculado como $\text{valorBase} * 0.05 + (\text{potenciaMotor} * 15.0)$.
6. **Crie a subclasse concreta Caminhao (herda de Veiculo):**
- **Atributo privado:** **quantidadeEixos** (int).
 - Construtor adequado repassando os dados comuns para a superclasse.
 - Sobrescreva o método **calcularLicenciamento()**: O valor do licenciamento é $\text{valorBase} * 0.08 + (\text{quantidadeEixos} * 200.0)$.
7. **Crie a classe executável TestaFrota (com método main):**
- Instancie dois objetos do tipo **CarroPasseio** e dois do tipo **Caminhao**.
 - Exiba os dados de cada veículo, o valor do licenciamento e a taxa ambiental.
 - Ao final, imprima o total de veículos cadastrados invocando o método estático **Veiculo.getTotalVeiculosCadastrados()**.

Questão 2: Sistema de Processamento Tributário e Auditoria (Interfaces e Classes Abstratas)

Crie um projeto chamado **atividade_interfaces_tributos**.

Imagine um sistema de gestão financeira em que diferentes produtos e contas bancárias devem calcular tributos e gerar relatórios de auditoria.

8. **Crie a interface Tributavel:**
- Método: **double calcularTributos()**.
9. **Crie a interface Auditavel:**
- Método: **String gerarRelatorioAuditoria()**.
10. **Crie a classe abstrata Conta:**
- **Atributos privados:** **numeroConta** (String), **titular** (String), **saldo** (double).
 - Construtor, métodos *getters/setters*, **depositar(double valor)** e **sacar(double valor)**.
 - Método abstrato: **public abstract void aplicarTaxaMensal()**.
11. **Crie a classe ContaCorrente (herda de Conta e implementa Tributavel e Auditavel):**
- Implemente **aplicarTaxaMensal()**: Desconta R\$ 15,00 do saldo.
 - Implemente **calcularTributos()**: Retorna 1% do saldo atual da conta.
 - Implemente **gerarRelatorioAuditoria()**: Retorna uma String formatada com o número da conta, titular, saldo e imposto calculado.
12. **Crie a classe SeguroDeVida (NÃO herda de Conta, mas implementa Tributavel e Auditavel):**
- **Atributos privados:** **numeroApolice** (String), **segurado** (String), **valorApolice** (double).
 - Implemente **calcularTributos()**: Retorna uma taxa fixa de R\$ 50,00.
 - Implemente **gerarRelatorioAuditoria()**: Retorna uma String com número da apólice, segurado, valor e tributo fixo.
13. **Crie a classe AuditoriaInterna:**
- **Atributo estático:** **totalTributosApurados** (double).

- Método **public void auditarEAcumular(Tributavel t)**: Calcula os tributos de **t**, soma ao **totalTributosApurados** e, caso o objeto também seja **instanceof Auditavel**, exibe o relatório de auditoria.
- Método estático **public static double getTotalTributosApurados()**.

14. **Crie a classe executável TestaAuditoria (main):**

- Instancie duas contas correntes e dois seguros de vida.
- Crie um objeto **AuditoriaInterna** e processe todos os objetos.
- Exiba o total de tributos arrecadados utilizando o método estático **AuditoriaInterna.getTotalTributosApurados()**.

Questão 3: Desafio Integrado — Sistema de Folha de Pagamento com Bonificação

Crie um projeto chamado **desafio_poo_integrado**.

Combine **Classes Abstratas, Métodos Abstratos, Membros Estáticos e Interfaces** em um único sistema de RH.

15. **Crie a interface Bonificavel:**

- Método: **double calcularBonificacaoAnual();**.

16. **Crie a classe abstrata Funcionario:**

- **Atributo estático privado: contadorMatricula** (int), iniciado em 100.
- **Atributo estático privado: pisoSalarial** (double), valor R\$ 1.412,00.
- **Atributos de instância privados: matricula** (int), **nome** (String), **salarioBase** (double).
- **Métodos:**
 - Construtor que gera a **matricula** automaticamente (**contadorMatricula++**) e garante que o **salarioBase** não seja inferior ao **pisoSalarial**.
 - Getters/Setters e método estático **public static void setPisoSalarial(double novoPiso)**.
 - Método abstrato **public abstract double calcularSalarioLiquido();**.

17. **Crie a subclasse Desenvolvedor (herda de Funcionario e implementa Bonificavel):**

- **Atributo privado: nivel** (String: "JUNIOR", "PLENO", "SENIOR").
- Implemente **calcularSalarioLiquido()**:
 - JUNIOR: **salarioBase * 0.90** (10% de descontos).
 - PLENO: **salarioBase * 0.85** (15% de descontos).
 - SENIOR: **salarioBase * 0.80** (20% de descontos).
- Implemente **calcularBonificacaoAnual()**:
 - JUNIOR: 1x o salário base.
 - PLENO: 2x o salário base.
 - SENIOR: 3x o salário base.

18. **Crie a subclasse GerenteProjeto (herda de Funcionario e implementa Bonificavel):**

- **Atributo privado: adicionalGerencial** (double).
- Implemente **calcularSalarioLiquido()**: **(salarioBase + adicionalGerencial) * 0.75**.
- Implemente **calcularBonificacaoAnual()**: **(salarioBase * 4) + adicionalGerencial**.

19. **Crie a classe executável TestaRH (main):**

- Crie um vetor ou lista de **Funcionario** contendo desenvolvedores de diferentes níveis e gerentes de projetos.
- Percorra a lista exibindo a matrícula automática, nome, salário líquido e a bonificação anual de cada colaborador.